

**SISTEMA DE REGISTRO Y GESTIÓN DE LLAMADAS DE EMERGENCIA PARA LA
REGIÓN DE TACNA**

Fabia Marien Torres Maquera 2026-119042

Alonso Salvador Luque Palomino 2026-119059

Giovanny Imanol Mamani Mamani 2026-119049



Universidad Nacional Jorge Basadre Grohomann

1er Ciclo A

Fundamentos de Programación

Dr. Hugo Manuel Barraza Vizcarra

15 de julio de 2026

INTRODUCCIÓN

El crecimiento urbano y la necesidad de optimizar los tiempos de respuesta ante las contingencias en la provincia de Tacna exigen sistemas de información eficientes de alta disponibilidad. El presente proyecto implementa un prototipo funcional denominado Sistema de Registro y Gestión de Llamadas de Emergencia para la Región de Tacna, desarrollado bajo los estándares de la programación estructurada y modular en lenguaje C++.

El sistema resuelve la necesidad de persistir y procesar datos dinámicamente en memoria RAM mediante el uso de Estructuras de Datos Homogéneas Lineales (Arreglos de Registros). La solución adopta una arquitectura Multi Archivo para separar la interfaz de usuario, la abstracción de datos y la lógica computacional del negocio, facilitando el desarrollo paralelo mediante el uso de Git y GitHub.

1. PLANTEAMIENTO DEL PROBLEMA

La centralización y velocidad de procesamiento de reportes de incidentes de la Región de Tacna enfrenta deficiencias debido a la falta de herramientas informáticas ligeras de escritorio que permitan a los operadores registrar de forma unificada las llamadas dirigidas a múltiples instituciones (Policía, Bomberos y Serenazgo).

Los sistemas tradicionales presentan un alto acoplamiento en su código, dificultando su escalabilidad y el mantenimiento algorítmico. Se requiere un software estructurado de alto rendimiento que permita realizar operaciones ágiles de creación, lectura, actualización y eliminación (CRUD) de datos sobre incidentes críticos, operando con eficiencia bajo restricciones severas de memoria estática.

2. OBJETIVO

Diseñar, codificar e implementar el Sistema de Registro y Gestión de Llamadas de Emergencia para la Región de Tacna bajo el paradigma estructurado modular en C++, empleando registros compuestos y funciones parametrizadas por referencia.

2.1 Objetivos específicos

- a. Modelar la abstracción de datos del mundo real mediante el uso de estructuras de datos heterogéneas (struct) según los lineamientos académicos de la Unidad 02.
- b. Implementar una distribución multiarchivo (.h y .cpp) para segregar las responsabilidades del sistema y garantizar una alta cohesión.
- c. Evidenciar el desarrollo colaborativo del equipo mediante la trazabilidad de commits

y gestión de ramas independientes en un repositorio centralizado de GitHub.

3. DESARROLLO DE LA PROPUESTA

3.1 Tecnologías de desarrollo

Para garantizar la viabilidad técnica, el rendimiento óptimo y la correcta gestión del ciclo de la vida del prototipo, se ha seleccionado un ecosistema tecnológico estandarizado para la ingeniería de software:

- a. **Lenguajes de programación:** C++ (Estandar ISO), elegido por su robustez, eficiencia en la gestión de memoria y el soporte nativo para la manipulación avanzada de tipos de datos definidos por el usuario (struct).
- b. **Entorno de Desarrollo Integrado (IDE):** Dev-C++, que provee las herramientas necesarias para la compilación, depuración y gestión de proyectos multiarchivos (.h y .cpp).
- c. **Sistema de Control de Versiones:** Git, integrado con la plataforma de alojamiento GitHub. Esta tecnología es un pilar crítico para coordinar el desarrollo colaborativo del equipo, permitiendo la integración de módulos de código mediante un historial de commits transparente y auditable.

3.2 Descripción de los Interesados y Usuarios

El sistema interactúa con diferentes actores dentro del ecosistema de atención de emergencias en la región de Tacna.

- a. **Operadores de la Central de Monitoreo:** Personal técnico encargado de la recepción de llamadas telefónicas ciudadanas. Requerirán un menú cíclico fluido y

de baja latencia por consola para ingresar datos rápidamente.

- b. **Supervisores de Incidentes de Tacna:** Usuarios con privilegios para consultar las listas consolidadas, auditar reportes específicos por ID o purgar llamadas duplicadas o erróneas del sistema.

3.3 Requerimientos Funcionales

- a. **RF-01 (Registrar Llamada):** El sistema debe capturar el nombre del ciudadano, zona de Tacna, institución de respuesta, hora y autogenerar una clave primaria numérica unívoca a partir del valor 1001.
- b. **RF-02 (Listar Emergencia):** El sistema debe recuperar iterativamente y desplegar la totalidad de celdas activas en el vector registros.
- c. **RF-03 (Buscar por ID):** El sistema debe ejecutar una búsqueda lineal indexada en base a un ID proporcionado por el usuario, interrumpiendo el bucle inmediatamente al encontrar la coincidencia.
- d. **RF-04 (Modificar Estado):** El sistema debe permitir la actualización dinámica en memoria del estado operacional de una llamada ("Pendiente", "En_Curso", "Atendido").
- e. **RF-05 (Eliminar Registro):** El sistema debe purgar de manera física un registro por ID, compactando el arreglo mediante un algoritmo de desplazamiento vectorial hacia la izquierda para evitar celdas nulas intermedias.

3.4 Arquitectura de Software

3.4.1 Lista de Datos Estructurados Utilizados

El sistema utiliza un registro estructurado principal encapsulado bajo el

preprocesador del sistema:

a. Estructura: struct Emergencia

- **idLlamada (int):** Clave primaria autogenerada unívoca para indexación y búsquedas secuenciales.
- **nombre (string):** Cadena identificadora del ciudadano reportante (capturada sin espacios).
- **zona (string):** Distrito o localización geográfica del suceso dentro de la región de Tacna.
- **institucion (string):** Unidad de respuesta asignada (Policía / Bomberos / Serenazgo).
- **hora (string):** Formato de tiempo de veinticuatro horas (HH:MM) del registro del suceso.
- **estado (string):** Flag de ciclo de vida del incidente (Pendiente por defecto, En Curso o Atendido).

3.4.2 Listas de Funciones Clasificadas por Áreas:.

a. Área de Inserción y Escritura (Módulo Create)

- `void registrarLlamada(Emergencia lista[], int &contador);`

```
// 1. REGISTRAR LLAMADA (Create)
#include <string>
#include "emergencia.h"
using namespace std;
void registrarLlamada(Emergencia lista[], int &contador) {
    if (contador >= 100) {
        cout << "\n[!] Almacenamiento lleno.\n";
        return;
    }
    cout << "\n--- REGISTRAR EMERGENCIA ---\n";
    lista[contador].idLlamada = 1000 + contador + 1;
    cout << "Nombre del ciudadano (sin espacios): ";
    cin >> lista[contador].nombre;
    cout << "Zona o Distrito de Tacna: ";
    cin >> lista[contador].zona;
    cout << "Institucion (Policia/Bomberos/Serenazgo): ";
    cin >> lista[contador].institucion;
    cout << "Hora del suceso (HH:MM): ";
    cin >> lista[contador].hora;

    lista[contador].estado = "Pendiente";
    contador++;
    cout << "\n[OK] Emergencia guardada con ID: " << lista[contador-1].idLlamada << endl;
}
```

b. Área de Consulta e Inspección Dinámica (Módulo Read)

- void mostrarLlamadas(const Emergencia lista[], int contador);

```
// 2. LISTAR EMERGENCIAS (Read)
void mostrarLlamadas(const Emergencia lista[], int contador) {
    if (contador == 0) {
        cout << "\n[!] No hay registros guardados.\n";
        return;
    }
    cout << "\n--- LISTA GENERAL DE INCIDENTES ---\n";
    for(int i = 0; i < contador; i++) {
        cout << "ID: " << lista[i].idLlamada
            << " | Ciudadano: " << lista[i].nombre
            << " | Zona: " << lista[i].zona
            << " | Unidad: " << lista[i].institucion
            << " | Hora: " << lista[i].hora
            << " | Estado: " << lista[i].estado << endl;
    }
}
```

- void buscarEmergencia(const Emergencia lista[], int contador);

```
// 3. BUSCAR POR ID (Read)
void buscarEmergencia(const Emergencia lista[], int contador) {
    if (contador == 0) {
        cout << "\n[!] Sistema vacio.\n";
        return;
    }
    int idBuscado;
    bool encontrado = false;
    cout << "\nIngrese el ID de la emergencia a buscar: ";
    cin >> idBuscado;

    for(int i = 0; i < contador; i++) {
        if(lista[i].idLlamada == idBuscado) {
            cout << "\nRegistro Encontrado\n";
            cout << "ID: " << lista[i].idLlamada << "\nCiudadano: " << lista[i].nombre
                << "\nZona: " << lista[i].zona << "\nUnidad: " << lista[i].institucion
                << "\nHora: " << lista[i].hora << "\nEstado: " << lista[i].estado << endl;
            encontrado = true;
            break;
        }
    }
    if(!encontrado) cout << "\n[!] No se encontro ninguna llamada con ese ID.\n";
}
```

c. Área de Edición y Actualización de Atributos (Módulo Update)

- void modificarEstado(Emergencia lista[], int contador);

```
// 4. MODIFICAR ESTADO (Update)
void modificarEstado(Emergencia lista[], int contador) {
    if (contador == 0) {
        cout << "\n[!] No hay llamadas para modificar.\n";
        return;
    }
    int idBuscado;
    cout << "\nIngrese el ID de la llamada a actualizar: ";
    cin >> idBuscado;

    for(int i = 0; i < contador; i++) {
        if(lista[i].idLlamada == idBuscado) {
            int opcEstado;
            cout << "\nEstado actual: " << lista[i].estado << endl;
            cout << "Seleccione nuevo estado: 1. En_Curso 2. Atendido\nOpcion: ";
            cin >> opcEstado;
            if(opcEstado == 1) lista[i].estado = "En_Curso";
            else if(opcEstado == 2) lista[i].estado = "Atendido";

            cout << "\n[OK] Estado actualizado correctamente.\n";
            return;
        }
    }
    cout << "\n[!] El ID especificado no existe.\n";
}
```

d. Área de Reestructuración Física de Memoria (Módulo Delete)

- void eliminarLlamada(Emergencia lista[], int &contador);

```
// 5. ELIMINAR LLAMADA (Delete)
void eliminarLlamada(Emergencia lista[], int &contador) {
    if (contador == 0) {
        cout << "\n[!] Nada que eliminar.\n";
        return;
    }
    int idBuscado;
    cout << "\nIngrese el ID del registro a eliminar: ";
    cin >> idBuscado;

    for(int i = 0; i < contador; i++) {
        if(lista[i].idLlamada == idBuscado) {
            for(int j = i; j < contador - 1; j++) {
                lista[j] = lista[j + 1];
            }
            contador--;
            cout << "\n[OK] Registro eliminado con exito del sistema.\n";
            return;
        }
    }
    cout << "\n[!] No se encontro el ID proporcionado.\n";
}
```

3.4.3 Diagramas de Flujo y descripción de funciones:

a. Descripción del Flujo Principal del Sistema: El Sistema de Gestión de

Llamadas de Emergencia (SGLE)-Tacna está estructurado bajo una arquitectura modular multi archivo (**main.cpp**, **emergencia.cpp** y **emergencia.h**).

El Flujo Principal (**controlado en el main.cpp**) funciona mediante un ciclo repetitivo do-while que mantiene el sistema activo y despliega un menú interactivo en la consola. El flujo sigue el siguiente orden lógico:

- **Inicialización:** Se declara el arreglo de estructuras (**struct Emergencia**) y un contador entero inicializado en 0 que controla el índice de registro en memoria.
- **Despliegue y Captura:** Se muestra el menú de opciones en pantalla y se captura la selección del usuario mediante un filtro de validación.
- **Derivación (Estructura de Control):** Utilizando una estructura condicional switch-case, el sistema evalúa la opción elegida. si la opción es válida, invoca

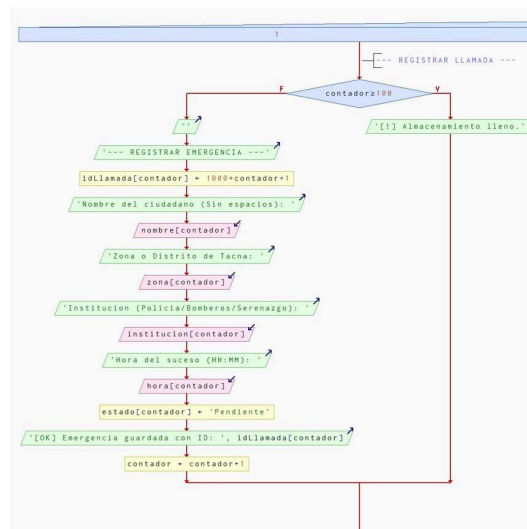
la función correspondiente alojada en **emergencia.cpp**, pasándole como parámetros el arreglo y el contador por referencia (cuando requiere modificarlo).

- **Cierre Seguro:** El ciclo solo se rompe si el usuario selecciona explícitamente la opción de “**Salir**”, liberando la memoria intermedia y finalizando la ejecución del programa de forma limpia.

b. Descripción Detallada de las Funciones del Módulo (emergencia.cpp): A

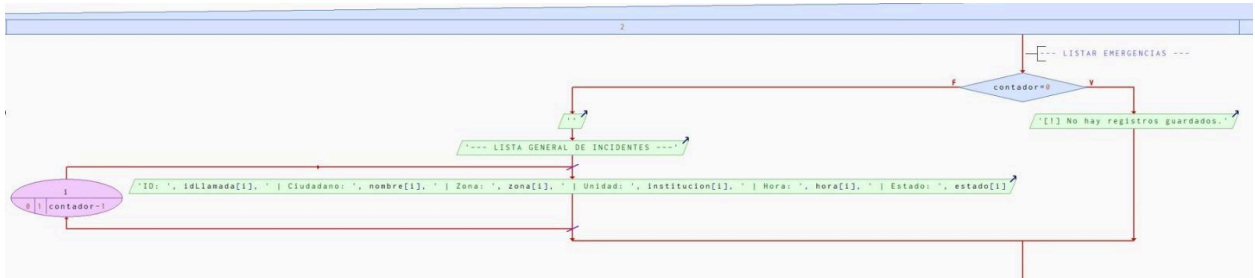
continuación, se describen las funciones que componen la lógica del negocio del proyecto:

- **void registrarLlamada(Emergencia lista[], int &contador):** Valida que el arreglo no supere los 100 registros. Genera automáticamente con ID único ($1000 + \text{contador} + 1$), solicita los datos de la emergencia (ciudadano, zona, institución y hora), asigna el estado inicial como “Pendiente” e incrementa el contador por referencia en + 1.

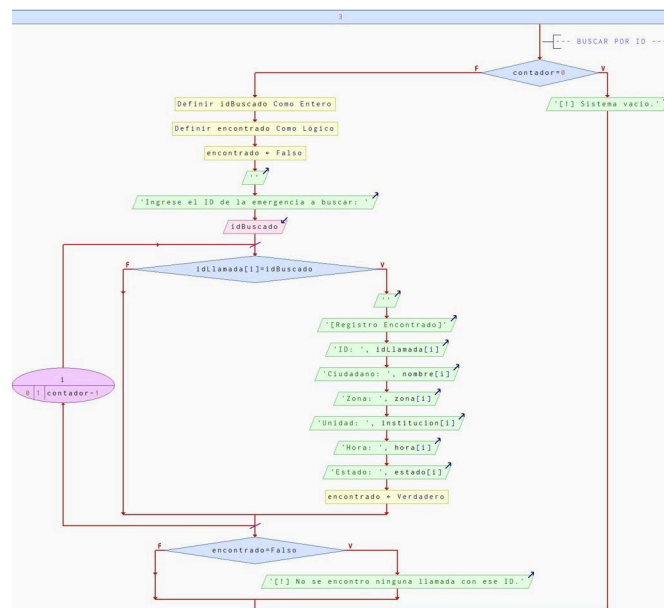


- **void mostrarLlamadas(const Emergencia lista[], int contador):** Módulo de lectura protegido con **const**. Verifica si el sistema está vacío; de lo contrario

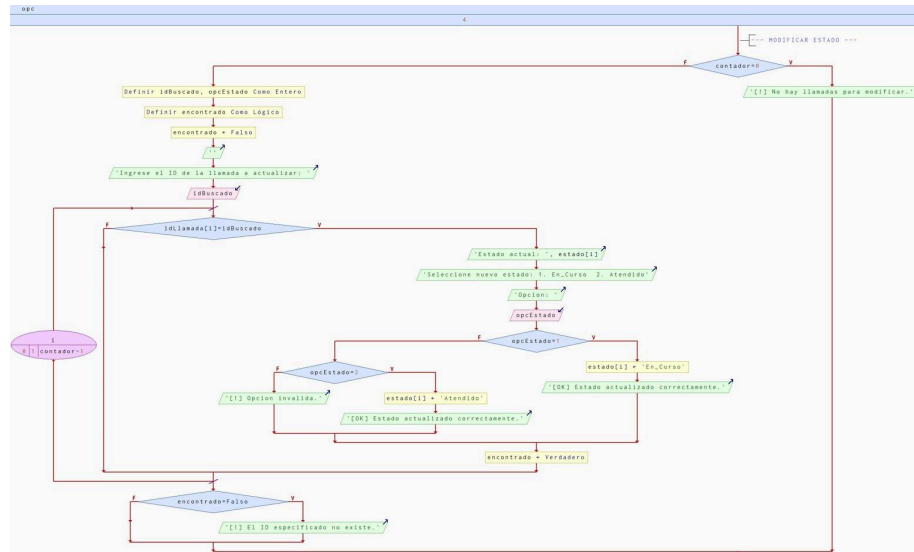
recorre el arreglo con un bucle **for** e imprime en consola el historial completo de incidentes (ID, ciudadano, zona, unidad, hora y estado) de forma ordenada.



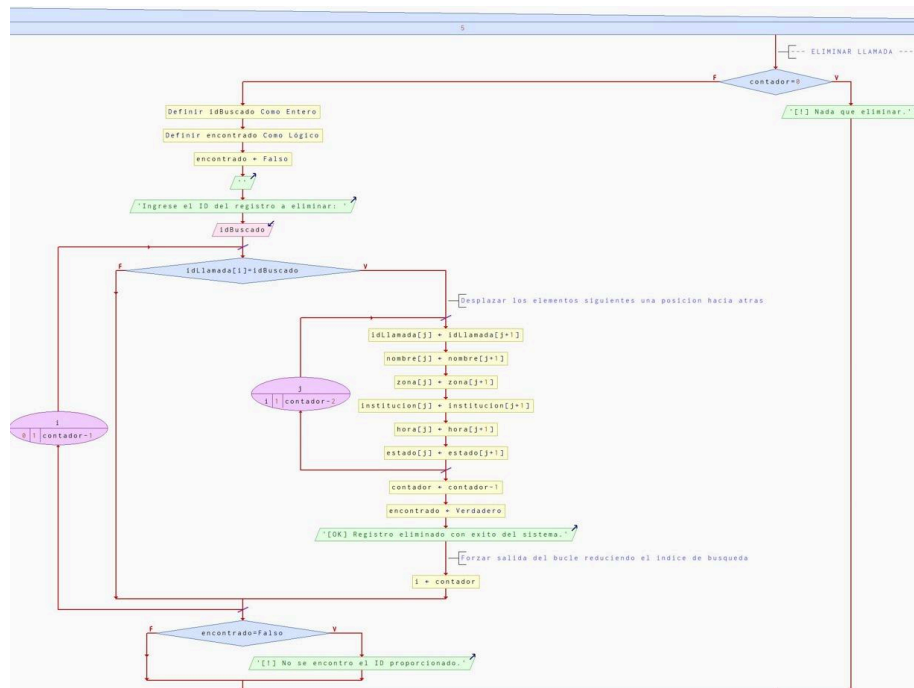
- **void buscarEmergencia(const Emergencia lista[], int contador):** Busca un incidente específico mediante una consulta por ID ingresado por el usuario. Utiliza un recorrido secuencial **for** y, al hallar la coincidencia, detiene el ciclo y despliega en pantalla la ficha técnica detallada del registro.



- **void modificarEstado(Emergencia lista[], int contador):** Permite la actualización operativa de un reporte. Escanea el arreglo buscando el ID solicitado y, tras localizarlo, despliega un submenú para conmutar el estado del incidente entre “En_curso” o “Atendido”, guardando el cambio en memoria.



- **void eliminarLlamada(Emergencia lista[], int &contador):** Se encarga de dar de baja un registro. Ubica el ID solicitado y, mediante el bucle anidado, descalza los elementos posteriores hacia la izquierda para sobrescribir el vacío y mantener los datos continuos, decrementando **contador** por referencia en - 1.



3.4.4 Capturas de Menús de Opciones Más importantes):

A continuación, se presentan las evidencias del correcto funcionamiento del Sistema de Gestión de Llamadas de Emergencia (SGLE) - Tacna ejecutado desde la terminal de consola. Las capturas demuestran la interactividad del menú principal, la consistencia de la estructura de datos en memoria intermedia y el flujo correcto de validación en cada uno de los módulos principales.

a. Interfaz del Menú Principal del Sistema:

Muestra el despliegue inicial de la pantalla con las 6 opciones disponibles del programa, esperando la interacción del operador de la central.

```
=====
CENTRAL CENTRALIZADA DE EMERGENCIAS - TACNA
=====
1. Registrar Llamada de Emergencia (Create)
2. Listar Todas las Emergencias (Read)
3. Buscar Emergencia por ID (Read)
4. Modificar Estado de la Emergencia (Update)
5. Eliminar Registro de Emergencia (Delete)
6. Salir del Sistema
=====
Seleccione una opcion (1-6):
```

b. Módulo de Registro de Incidentes (registrarLlamadas):

Evidencia la captura de datos de un ciudadano. Se observa la asignación automática del ID único y el estado inicial asignado por defecto como “Pendiente”

```
=====
Seleccione una opcion (1-6): 1
--- REGISTRAR EMERGENCIA ---
Nombre del ciudadano (Sin_espacios): FabiaTorres
Zona o Distrito de Tacna: Cercado
Institucion (Policia/Bomberos/Serenazgo): Policia
Hora del suceso (HH:MM): 21:29

[OK] Emergencia guardada con ID: 1001
```

c. Módulo de Reporte General (mostrarLlamadas):

Visualización de la lista de emergencias. Demuestra que el arreglo recorre los datos de forma limpia y los muestra ordenados en pantalla.

```
=====
Seleccione una opcion (1-6): 2
--- LISTA GENERAL DE INCIDENTES ---
ID: 1001 | Ciudadano: FabiaTorres | Zona: Cercado | Unidad: Policia | Hora: 21:29 | Estado: Pendiente
```

d. Módulo de Consulta de Incidentes (buscarEmergencia):

Muestra el proceso de búsqueda secuencial por ID único y el despliegue de la ficha técnica con todos los datos detallados del registro.

```
=====
Seleccione una opcion (1-6): 3
Ingrese el ID de la emergencia a buscar: 1001
[Registro Encontrado]
ID: 1001
Ciudadano: FabiaTorres
Zona: Cercado
Unidad: Policia
Hora: 21:29
Estado: Pendiente
```

e. Módulo de Actualización Operativa (modificarEstado):

Demuestra el proceso donde el operador digita el ID de la llamada y selecciona mediante el submenú el nuevo estado del incidente (En curso o Atendido).

```
=====
Seleccione una opcion (1-6): 4
Ingrese el ID de la llamada a actualizar: 1001
Estado actual: Pendiente
Seleccione nuevo estado: 1. En_Curso 2. Atendido
Opcion: 2
[OK] Estado actualizado correctamente.
```

f. Módulo de Baja de Registros (eliminarLlamada):

Evidencia la remoción exitosa de un reporte del sistema desplazando los elementos en memoria, junto con el mensaje de finalización del programa al seleccionarla opción de salida.

```
=====
Seleccione una opcion (1-6): 5
Ingrese el ID del registro a eliminar: 1001
[OK] Registro eliminado con exito del sistema.
```

4. METODOLOGÍA DE TRABAJO

Para dar cumplimiento estricto a las exigencias tecnológicas de la escuela, el desarrollo se distribuyó mediante el control de versiones asíncrono:

- c. Estrategias de Ramificación:** Se bloquearon las interacciones directas sobre la rama main. En su lugar, Fabia asignó sub-ramas especializadas independientes a cada desarrollador para segmentar el avance: desarrollo-emergencia-h para el diseño estructural (Giovanny), y desarrollo-emergencia-cpp para la algoritmia secuencial pura (Alonso).
- d. Gestión de Incidencias e Integración:** Durante las pruebas en caliente, el equipo resolvió errores críticos de enlazado (linker) causados por la doble extensión física de archivos ocultos en Windows (emergencia.cpp.cpp) y la omisión del archivo objeto en el árbol binario de dependencias de Dev-C++, solucionado satisfactoriamente mediante la inyección explícita utilizando el comando "Añadir al Proyecto".

5. CONCLUSIONES

Se implementó con éxito una arquitectura de software modular multi archivo

(**main.cpp**, **emergencia.cpp** y **emergencia.h**) que organiza los reportes mediante un arreglo de estructuras (**struct Emergencia**). Esto optimiza la gestión de datos directamente.

El uso del calificador const en las funciones de visualización y búsqueda secuencial (**mostrarLlamadas** y **buscarEmergencia**) demostró ser clave para proteger la integridad de los datos, impidiendo alteraciones accidentales en memoria mientras se realiza el control y la actualización de los estados de cada incidente de forma segura.

El diseño algorítmico de la función **eliminarLlamada** resuelve eficientemente la baja de registros mediante un bucle de desplazamiento secuencial hacia la izquierda. Esto elimina los espacios vacíos, mantiene la contigüidad física del arreglo en memoria intermedia y actualiza correctamente el tamaño del vector mediante el manejo del contador por referencia.